User Manual

Adaptive Mesh Refinement Tool for Immersed Boundary Methods

Dr.-Ing. Jonatan Núñez de la Rosa

20th August 2026

Adaptive Mesh Refinement Tool for Immersed Boundary Methods

1. General Information

AMR-for-IBM is a new preprocessing tool for mesh adaptation on hybrid unstructured meshes with a target application on immersed boundary methods. The tool has as input an unstructured, hybrid, and conforming mesh generated by an external mesh generation software, and the main goal is to refine this mesh around immersed geometries in such a way that the CFD solver using the immersed boundary method can simulate flow problems in an accurate and efficient manner. The input background mesh can be made of different types of elements, like tetrahedra, hexahedra, prisms, and pyramids, which, unlike Cartesian meshes, permit for a more flexible mesh.

Name	AMR-for-IBM
Version	v1.0.0
Developer	Dr.-Ing. Jonatan Núñez de la Rosa
License	GPLv3 (GNU General Public License, version 3)
Repository	https://github.com/jbnunezd/mesh-refinement-for-ibm
Programming Language	Modern Fortran
Operating System	GNU Linux
External Libraries	HDF5
Description	Mesh refinement tool for immersed boundary methods

2. Installation

2.1. Software Requirements

AMR-for-IBM is written in Modern Fortran programing language. The program has been tested in systems with GNU Linux. The project is built with CMake and requires the following software/libraries to be compiled:

- Fortran compiler (e.g., gfortran version 12.5 or newer)
- CMake (version 3.20 or newer)
- HDF5 library (version 1.10.8 or newer)

2.2. Software Compilation

AMR-for-IBM can obtained by downloading the source code from its GitHub repository:

Cloning the repository

```
git clone https://github.com/jbnunezd/mesh-refinement-for-ibm.git
```

Next, the source code is built using CMake:

Building the executable

```
cd mesh-refinement-for-ibm
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Running an example can be done by executing the following instruction:

Running an example

```
./bin/mesh-refinement-for-ibm ./ini/parameter.ini
```

3. Structure of the Configuration File

In the configuration file the input parameters are written in the *key = value* format. The admitted data types are listed below:

```
Par1 = "method"          # scalar parameter of CHARACTER type
Par2 = 123               # scalar parameter of INTEGER type
Par3 = 1.23E4            # scalar parameter of REAL type
Par4 = .TRUE.            # scalar parameter of LOGICAL type
Par5 = (/1,2,3/)         # vector parameter of INTEGER type
Par6 = (/1.0,2.0,3.0/)   # vector parameter of REAL type
```

The parameters that can be used in the configuration file are listed below, classified according to the modules implemented in the main program.

3.1. Project Information

- **ProjectName**: name of the project.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: a valid project name.

3.2. Mesh Construction Method

- **NGeo**: degree of the mesh.
  Data type: scalar parameter of INTEGER type.
  Values accepted: 1.
- **MeshDimension**: dimension of the mesh.
  Data type: scalar parameter of INTEGER type.
  Values accepted: 2 and 3.
- **WhichMeshConstructionMethod**: method used to build the mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "mesh-builtin", and "mesh-import".

Construction method: mesh-builtin

- **WhichMeshType**: built-in type of mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "cartesian-domain".
- **WhichOutputBasis**: built-in output basis used in the generation of the mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "uniform", "chebyshev-gauss", "chebyshev-gauss-lobatto", "legendre-gauss", "legendre-gauss-lobatto".
- **WhichMapping2D**: built-in mapping used in the generation of the two-dimensional mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "straight-quad".
- **WhichMapping3D**: built-in mapping used in the generation of the three-dimensional mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "straight-hexa".
- **StretchMesh**: built-in flag for stretching the mesh.
  Data type: scalar parameter of LOGICAL type.
  Values accepted: .TRUE. and .FALSE..
- **WhichMeshStretching**: built-in function used to stretch the mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "uniform", "geometric-progression".
- **MeshStretchingFactor2D**: stretching factor in x-, and y-directions.
  Data type: vector parameter of REAL type.
  Values accepted: real values larger than zero.
- **MeshStretchingFactor3D**: stretching factor in x-, y-, and z-directions.
  Data type: vector parameter of REAL type.
  Values accepted: real values larger than zero.
- **WarpMesh**: built-in flag to warp the mesh.
  Data type: scalar parameter of LOGICAL type.
  Values accepted: .TRUE. and .FALSE..
- **MeshDeformationFactor**: factor used by the warp function to warp the mesh.
  Data type: scalar parameter of REAL type.
  Values accepted: real values larger than zero.
- **nBoxElems2D**: number of elements of the two-dimensional mesh in x- and y-directions.
  Data type: vector parameter of INTEGER type.
  Values accepted: integer values larger than zero.
- **nBoxElems3D**: number of elements of the three-dimensional mesh in x-, y-, and z-directions.
  Data type: vector parameter of INTEGER type.
  Values accepted: integer values larger than zero.
- **BoxCorner**: coordinates of the 4/8 points defining the quadrilateral/hexahedral domain. Points ordering obeys the CGNS convention for a quadrilateral and hexahedron. Each point of the quadrilateral/hexahedral has to be provided separately.
  Data type: vector parameter of REAL type.
  Values accepted: real values.

Construction method: mesh-import

- **InputMeshFolder**: folder where the input mesh file is located.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: a valid folder name.
- **InputMeshFile**: name of the input mesh file.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: a valid file name.
- **InputMeshFileFormat**: format of the input mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "gmsh" (file version 2.2, ASCII format).

Common Parameters

- **RotateMesh**: flag for rotating the mesh.
  Data type: scalar parameter of LOGICAL type.
  Values accepted: .TRUE. and .FALSE..
- **RotationAngle2D**: rotation angle of the two-dimensional mesh.
  Data type: scalar parameter of REAL type.
  Values accepted: real values.
- **RotationAngle3D**: rotation angle of the three-dimensional mesh.
  Data type: vector parameter of REAL type.
  Values accepted: real values.
- **BoundaryMark**: tag or mark of the boundary condition
  Data type: scalar parameter of INTEGER type.
  Values accepted: integer values accordingly to the boundary conditions.
- **BoundaryName**: name of the boundary conditions
  Data type: scalar parameter of CHARACTER type. Format of the input mesh.
  Values accepted: a valid boundary condition name.
- **nRefinedBox**: number of box regions to be refined.
  Data type: scalar parameter of INTEGER type.
  Values accepted: integer value larger or equal to zero.
- **RefinedBoxName**: name of the refined region.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: a valid name.
- **RefinedBoxCorner**:
  Data type: coordinates of the 4/8 points of the quadrilateral/hexahedral box regions. Points ordering obeys the CGNS convention for a quadrilateral and hexahedron. Each point of the quadrilateral/hexahedron has to be provided separately.
  Data type: vector parameter of REAL type.
  Values accepted: real values.

3.3. Input Geometry

- **InputGeometryFolder**: folder where the input geometry file is located.
  Data type: scalar parameter of CHARACTER type.

Values accepted: a valid folder name.

- **InputGeometryFile**: name of the input geometry file.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: a valid file name.
- **InputGeometryFileFormat**: format of the input geometry file.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "stl" (ASCII or binary).

3.4. Mesh Refinement

- **WhichMeshRefinement**: method used to refine the mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "refine-all-elements", "refine-elements-inside-box", "refine-elements-around-geometry", "refine-elements-around-geometry-and-inside-box".
- **WhichBoxedRegion**: type of boxed region for refinement.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "standard-box", "adaptive-box".
- **WhichBodyGeometry**: type of immersed geometry.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "built-in-geometry", "imported-geometry".
- **WhichGeometryDistribution**: type of distribution of the geometry.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "stl-facets-distribution", "stl-points-distribution".
- **TessellationMaxEdgeLength**: characteristic size of the geometry tessellation.
  Data type: scalar parameter of REAL type.
  Values accepted: real values larger than zero.
- **ElementEnlargementFactor**: factor used by the box-triangle-overlap function to increase the size of corresponding box during the testing.
  Data type: scalar parameter of REAL type.
  Values accepted: real values larger than zero.
- **RegionEnlargementFactor**: factor used to increase the size of the boxed region to be refined.
  Data type: scalar parameter of REAL type.
  Values accepted: real values larger than zero.
- **MaxRefinementLevel**: maximum refinement level around the immersed geometry.
  Data type: scalar parameter of INTEGER type.
  Values accepted: integer value larger or equal to zero.
- **MaxBoxRefinementLevel**: maximum refinement level for each refined box region.
  Data type: vector parameter of INTEGER type.
  Values accepted: integer values larger or equal to zero.
- **IsotropicRefinement**: flag for refining the mesh into all directions.
  Data type: scalar parameter of LOGICAL type.
  Values accepted: .TRUE. and .FALSE..

3.5. Data Output

- **ExportMesh**: flag to export the mesh.
  Data type: scalar parameter of LOGICAL type.
  Values accepted: .TRUE. and .FALSE..
- **MeshExportFormat**: format of the output mesh.
  Data type: scalar parameter of CHARACTER type.
  Values accepted: "hdf5-coda", "tecplot-ascii", "gmsh" (file version 2.2, ASCII format).

4. Examples

4.1. Built-in mesh: NACA0012 airfoil

In this example, an initial background mesh is created by the preprocessing tool, and this is a hexahedral block made of $40 \times 1 \times 40$ elements. A box region is created inside the computational domain and further refined up to level $l = 5$. After this refinement, the input geometry is distributed in the computational mesh and then refined up to level $l = 8$. The initial mesh has 1.6×10^3 elements and the final mesh has 1.85×10^4 elements. The resulting mesh is depicted in Figure 1.

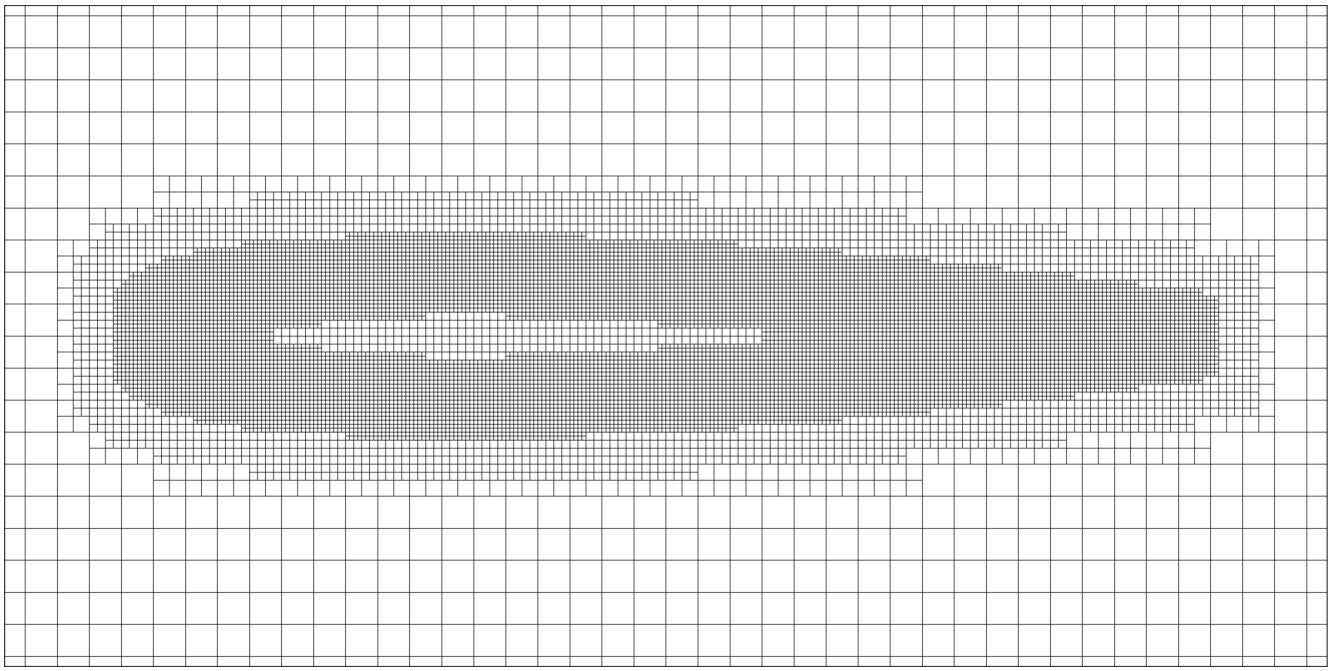

Figure 1: Refined mesh around an NACA0012 profile. The initial background mesh is a hexahedral block created by the preprocessing tool.

parameter-builtin-naca0012.ini

```
# PROJECT NAME
ProjectName = "TEST-01"
# PREPROCESSING METHOD
NGeo = 1
MeshDimension = 3
WhichMeshConstructionMethod = "mesh-builtin"
# MESH REFINEMENT AROUND GEOMETRY
```

```
InputGeometryFile       = "naca0012-airfoil.stl"
InputGeometryFolder     = "./ini/geometries"
InputGeometryFileFormat = "stl"
# MESH BUILT-IN
WhichMeshType          = "cartesian-domain"
WhichOutputBasis       = "uniform"
WhichMapping3D         = "straight-hexa"
StretchMesh            = .FALSE.
WhichMeshStretching    = "uniform"
MeshStretchingFactor3D = (/1.5,1.5,1.5/)
WarpMesh               = .FALSE.
MeshDeformationFactor  = 0.125
# MESH ROTATION
RotateMesh      = .FALSE.
RotationAngle3D = (/90.0,0.0,0.0/)
# COMPUTATIONAL DOMAIN
nBoxElems3D = (/40,1,40/)
BoxCorner = (/-20.0,+0.0,-20.0/)
BoxCorner = (/+20.0,+0.0,-20.0/)
BoxCorner = (/+20.0,+1.0,-20.0/)
BoxCorner = (/-20.0,+1.0,-20.0/)
BoxCorner = (/-20.0,+0.0,+20.0/)
BoxCorner = (/+20.0,+0.0,+20.0/)
BoxCorner = (/+20.0,+1.0,+20.0/)
BoxCorner = (/-20.0,+1.0,+20.0/)
# MESH REFINEMENT INSIDE BOX
nRefinedBox = 1
RefinedBoxName = "airfoil"
RefinedBoxCorner = (/-1.0,+0.0,-0.5/)
RefinedBoxCorner = (/+2.0,+0.0,-0.5/)
RefinedBoxCorner = (/+2.0,+1.0,-0.5/)
RefinedBoxCorner = (/-1.0,+1.0,-0.5/)
RefinedBoxCorner = (/-1.0,+0.0,+0.5/)
RefinedBoxCorner = (/+2.0,+0.0,+0.5/)
RefinedBoxCorner = (/+2.0,+1.0,+0.5/)
RefinedBoxCorner = (/-1.0,+1.0,+0.5/)
# BOUNDARY CONDITIONS
BoundaryMark = 1
BoundaryName = "bottom"
BoundaryMark = 2
BoundaryName = "front"
BoundaryMark = 3
BoundaryName = "right"
BoundaryMark = 4
BoundaryName = "back"
BoundaryMark = 5
BoundaryName = "left"
BoundaryMark = 6
BoundaryName = "top"
# MESH REFINEMENT
WhichMeshRefinement       = "refine-elements-around-geometry-and-inside-box"
WhichBoxedRegion          = "adaptive-box"
WhichBodyGeometry         = "imported-geometry"
WhichGeometryDistribution = "stl-facets-distribution"
TessellationMaxEdgeLength = 1.0E-02
ElementEnlargementFactor  = 10.0
RegionEnlargementFactor   = 0.1
MaxRefinementLevel        = 8
MaxBoxRefinementLevel     = (/5/)
IsotropicRefinement       = .FALSE.
# EXPORT MESH PARAMETERS
ExportMesh       = .TRUE.
MeshExportFormat = "hdf5-coda"
MeshExportFormat = "tecplot-ascii"
MeshExportFormat = "gmsh"
```

4.2. Imported mesh: NACA0012 airfoil

In this example, an initial background mesh is created by the GMSH software and is made of hexahedral elements. A box region is created inside the computational domain and further refined up to level $l = 5$. After this refinement, the input geometry is distributed in the computational mesh and then refined up to level $l = 8$. The initial mesh has 3.0×10^4 elements and the final refined mesh has 9.2×10^4 elements. The resulting mesh is depicted in Figure 2.

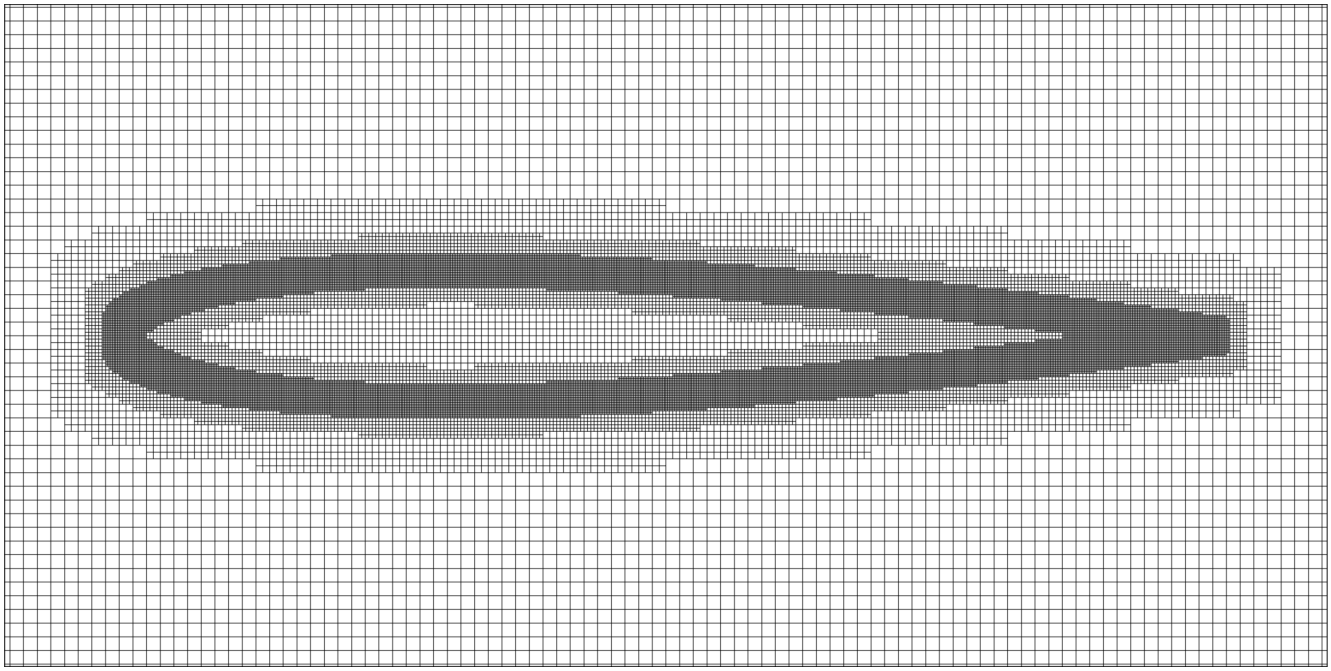

Figure 2: Refined mesh around an NACA0012 profile. The initial background mesh was created by the GMSH software and is made of hexahedral elements.

parameter-import-full-hexahedral.ini

```
# PROJECT NAME
ProjectName = "TEST-02"
# PREPROCESSING METHOD
NGeo = 1
MeshDimension = 3
WhichMeshConstructionMethod = "mesh-import"
# MESH REFINEMENT AROUND GEOMETRY
InputGeometryFile       = "naca0012-airfoil.stl"
InputGeometryFolder     = "./ini/geometries"
InputGeometryFileFormat = "stl"
# MESH IMPORT
InputMeshFolder     = "./ini/meshes"
InputMeshFile       = "mesh-full-hexahedral.msh"
InputMeshFileFormat = "gmsh"
# MESH ROTATION
RotateMesh = .FALSE.
RotationAngle3D = (/90.0,0.0,0.0/)
# MESH REFINEMENT INSIDE BOX
nRefinedBox = 1
RefinedBoxName = "airfoil"
RefinedBoxCorner = (/-1.0,+0.0,-0.5/)
RefinedBoxCorner = (/+2.0,+0.0,-0.5/)
RefinedBoxCorner = (/+2.0,+1.0,-0.5/)
RefinedBoxCorner = (/-1.0,+1.0,-0.5/)
RefinedBoxCorner = (/-1.0,+0.0,+0.5/)
```

```
RefinedBoxCorner = (/+2.0,+0.0,+0.5/)
RefinedBoxCorner = (/+2.0,+1.0,+0.5/)
RefinedBoxCorner = (/-1.0,+1.0,+0.5/)
# BOUNDARY CONDITIONS
BoundaryMark = 1
BoundaryName = "FarField"
BoundaryMark = 2
BoundaryName = "SymmetryPlane"
# MESH REFINEMENT
WhichMeshRefinement        = "refine-elements-around-geometry-and-inside-box"
WhichBoxedRegion           = "adaptive-box"
WhichBodyGeometry          = "imported-geometry"
WhichGeometryDistribution = "stl-facets-distribution"
TessellationMaxEdgeLength = 1.0E-02
ElementEnlargementFactor  = 10.0
RegionEnlargementFactor    = 0.1
MaxRefinementLevel         = 8
MaxBoxRefinementLevel      = (/5/)
IsotropicRefinement        = .FALSE.
# EXPORT MESH PARAMETERS
ExportMesh       = .TRUE.
MeshExportFormat = "hdf5-coda"
MeshExportFormat = "tecplot-ascii"
MeshExportFormat = "gmsh"
```

4.3. Imported mesh: MDA30P30N multi-element airfoil

In this example, an initial background mesh is created by the GMSH software and is made of hexahedra and prisms elements. The input geometry is distributed in the computational mesh and then refined up to level $l = 3$. The initial mesh has 5.3×10^5 elements and the final refined mesh has 9.3×10^5 elements. The resulting mesh is depicted in Figure 3.

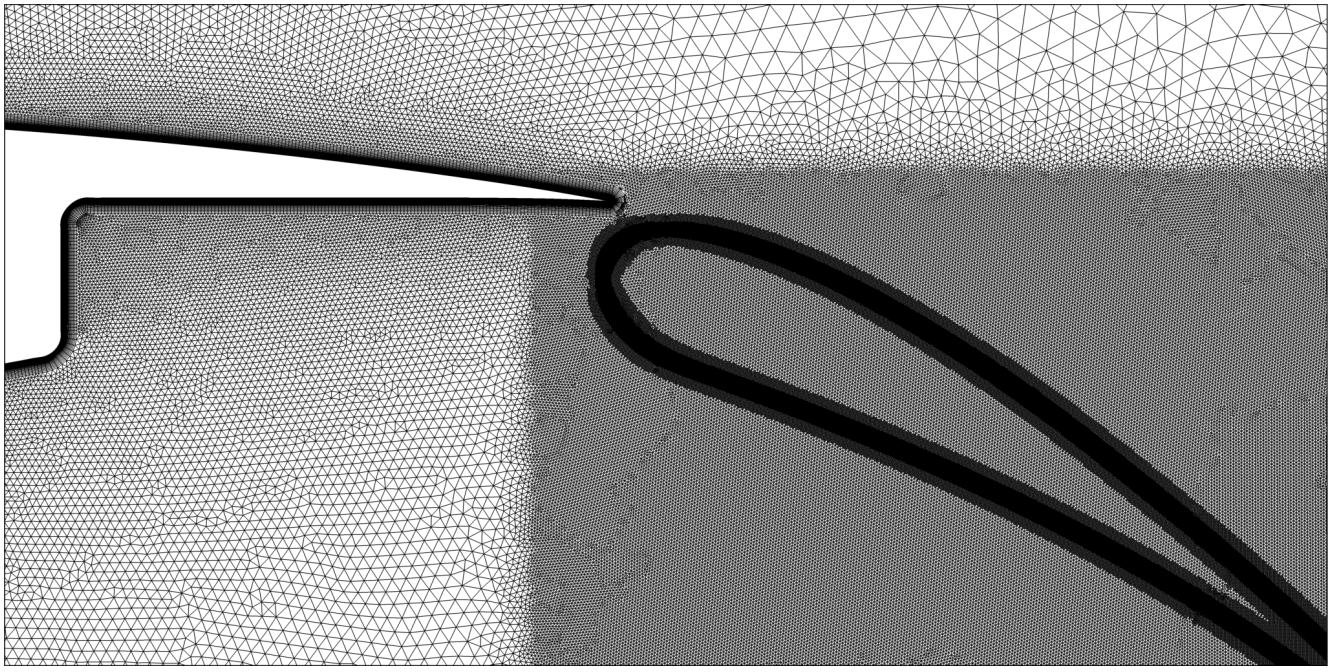

Figure 3: Refined mesh around an MDA30P30N multi-element airfoil. The initial background mesh was created by the GMSH software, and is a body-fitted mesh only for the slat and the main airfoil.

parameter-import-mda30p30n.ini

```
# PROJECT NAME
ProjectName = "TEST-03"
# PREPROCESSING METHOD
NGeo = 1
MeshDimension = 3
WhichMeshConstructionMethod = "mesh-import"
# MESH REFINEMENT AROUND GEOMETRY
InputGeometryFile       = "mda30p30n-flap.stl"
InputGeometryFolder     = "./ini/geometries"
InputGeometryFileFormat = "stl"
# MESH IMPORT
InputMeshFolder     = "./ini/meshes"
InputMeshFile       = "mda30p30n-wing-slat.msh"
InputMeshFileFormat = "gmsh"
# MESH ROTATION
RotateMesh = .FALSE.
RotationAngle3D = (/90.0,0.0,0.0/)
# BOUNDARY CONDITIONS
BoundaryMark = 1
BoundaryName = "FarField"
BoundaryMark = 2
BoundaryName = "SymmetryPlane"
BoundaryMark = 3
BoundaryName = "Wing"
BoundaryMark = 4
BoundaryName = "Slat"
# MESH REFINEMENT
WhichMeshRefinement       = "refine-elements-around-geometry-and-inside-box"
WhichBoxedRegion          = "adaptive-box"
WhichBodyGeometry         = "imported-geometry"
WhichGeometryDistribution = "stl-facets-distribution"
TessellationMaxEdgeLength = 1.0E-02
ElementEnlargementFactor  = 10.0
RegionEnlargementFactor   = 0.1
MaxRefinementLevel        = 3
IsotropicRefinement       = .FALSE.
# EXPORT MESH PARAMETERS
ExportMesh       = .TRUE.
MeshExportFormat = "hdf5-coda"
MeshExportFormat = "tecplot-ascii"
MeshExportFormat = "gmsh"
```

4.4. References

[1] Núñez-de la Rosa, Jonatan et al. *Implementation of immersed boundaries via volume penalization in the industrial aeronautical computational fluid dynamics solver CODA*. Engineering with Computers, 41(4):2571–2592, 2025.
DOI: 10.1007/s00366-025-02119-x

[2] Núñez-de la Rosa, Jonatan et al. *Mesh adaptation on hybrid unstructured meshes for immersed boundary methods*. Research paper submitted.
DOI: 10.48550/arXiv.2607.27580